Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

LAB MANUAL

Course No: CSE 2102

Course Name: Discrete Mathematics Sessional

Semester: 2nd Year Odd Semester

Prepared By:

Md. Azmain Yakin Srizon
Assistant Professor
Department of CSE, RUET

Rajshahi, Bangladesh

Last Updated: February 26, 2025

Discrete Mathematics Laboratory Guidelines

Before Coming to Lab

- Read the experiment sheet/manual thoroughly before each lab session.
- Revise the related mathematical concepts such as logic, set theory, and number theory.
- Prepare pseudocode or outline of the proof if the lab requires algorithmic or logical explanation.
- Bring notebook, calculator, and required materials for computation and reasoning.

During Lab Session

A. Conduct

- Be punctual; attendance will be recorded at the beginning.
- Maintain discipline and uphold academic integrity.
- Work with concentration and respect collaborative learning.

B. Work Process

- Perform all assigned problems and programming tasks individually or in assigned groups.
- Follow instructor's instructions and report any computational or logical discrepancy.
- Record every derivation, code snippet, and explanation neatly in your notebook.

After Completing Experiment

- Demonstrate your solution or program for evaluation.
- Submit the lab report including Objective, Theory, Algorithm/Proof, Code (if any), Output, and Conclusion.
- Maintain cleanliness and ensure code files and reports are submitted properly.

Evaluation Criteria

Criteria	Description
Preparation	Pre-lab readiness, understanding of mathematical concepts
Implementation	Correctness of logical proofs, code, and computational results
Understanding	Ability to explain reasoning, proofs, and derivations
Reporting	Completeness, clarity, and organization of the report
Discipline	Attitude, participation, and teamwork

CO-PO Mapping for CSE 2102

CO	CO Statement	Mapped PO	Assessment Tools
CO1	Apply principles of propositional logic and proof techniques to solve mathematical problems and construct logical proofs in practical scenarios.	PO1	Lab reports, code review, viva
CO2	Demonstrate practical applications of set theory, functions, and number theory in computational contexts.	PO1	Lab reports, discussion, viva
CO3	Utilize principles of mathematical induction, recursion, and counting to analyze and solve complex mathematical problems.	PO2	Lab reports, viva, problem analysis
CO4	Apply graph theory concepts to design and develop solutions for engineering problems.	PO3	Lab reports, quiz, presentation

Course Overview

Course Code: CSE 2102
Course Title: Discrete Mathematics Sessional
Credit: 1.50
Semester: 2nd Year Odd

Course Rationale

This course introduces students to discrete mathematical techniques that are fundamental to computer science and engineering. It emphasizes logical reasoning, proof construction, and mathematical modeling. Through practical problem solving, students develop skills in logic, set operations, number theory, induction, recursion, counting, and graph theory.

Course Objectives

- To apply propositional logic and proof techniques for solving discrete mathematical problems.
- To analyze sets, functions, sequences, and number theory through computational and algorithmic approaches.
- To implement and interpret mathematical induction, recursion, and combinatorial methods.
- To apply graph theory to model and solve real-world engineering problems.

Lab and Experiment Plan

Week	Topics	Delivery Methods	Aligned CO
1	Propositional Logic and Applications, Logical Equivalences	Lecture, Lab Manual, Discussion	CO1
2	Predicates, Quantifiers, and Rules of Inference	Lecture, Lab Manual, Interaction	CO1
3	Sets and Quantifiers, Generalized Unions and Intersections, Computer Representation of Sets	Lecture, Lab Report, Discussion	CO2
4	One-to-One and Onto Functions, Inverse and Composite Functions, Floor and Ceiling Functions	Lecture, Practice, Code Review	CO2
5	Sequences, Recurrence Relations, and Summations	Lecture, Problem Solving, Code Review	CO2
6	Countable and Uncountable Sets, Uncomputable Functions	Lecture, Discussion, Report Submission	CO2
7	Divisibility, Modular Arithmetic, and Greatest Common Divisors	Lecture, Manual, Discussion	CO2
8	Solving Congruences	Lecture, Lab Manual, Interaction	CO2
9	Mathematical Induction, Strong Induction, and Recursive Definitions	Lecture, Code Practice, Discussion	CO3
10	Counting Principles, Pigeonhole Principle, Binomial Coefficients, and Combinations	Lecture, Lab Manual, Quiz	CO3
11	Graph Models and Terminology, Graph Representations, Graph Traversal	Lecture, Interaction, Lab Report	CO4
12	Graph Isomorphism, Connectivity, Euler and Hamilton Paths	Lecture, Discussion, Code Practice	CO4
13	Planar Graphs and Graph Coloring	Lecture, Lab Report, Presentation	CO4
14	Revision Lab and Comprehensive Viva Preparation	Review, Quiz, Oral Evaluation	All

Assessment and Marks Distribution

- **Continuous and Summative Assessment: 75%**
  Lab reports, code reviews, and quizzes across CO1–CO4.
- **Board Viva: 25%**
  Final oral assessment based on overall understanding and performance.

Learning Materials

1. Kenneth H. Rosen, *Discrete Mathematics and Its Applications.*
2. Eric Lehman, F. Thomson Leighton, and Albert R. Meyer, *Mathematics for Computer Science.*
3. Ronald L. Graham, Donald E. Knuth, and Oren Patashnik, *Concrete Mathematics: A Foundation for Computer Science.*
4. S. G. Telang, *Number Theory.*

Generic Skills Developed

- Logical and Analytical Thinking
- Mathematical Problem-Solving Skills
- Abstract and Critical Reasoning
- Quantitative and Computational Skills
- Graphical Modeling and Representation

Learning Materials

1. Kenneth H. Rosen, *Discrete Mathematics and Its Applications.*
2. Eric Lehman, F. Thomson Leighton, and Albert R. Meyer, *Mathematics for Computer Science.*
3. Ronald L. Graham, Donald E. Knuth, and Oren Patashnik, *Concrete Mathematics: A Foundation for Computer Science.*

Generic Skills Developed

- Logical and Analytical Thinking
- Mathematical Problem Solving
- Proof Construction and Verification
- Quantitative and Computational Reasoning
- Application of Discrete Models to Engineering Problems

Lab 1: Propositional Logic and Applications

Objective

To understand the fundamentals of propositional logic and verify logical equivalences through truth tables and automated evaluation using Python.

Theory

Propositional logic forms the foundation of reasoning in computer science, enabling evaluation of conditions, decision-making, and logical circuit design. A **proposition** is a declarative statement that is either true (T) or false (F).

1. Logical Connectives

Common logical operators and their meanings:

$$\neg p : \text{NOT } p, \qquad p \wedge q : p \text{ AND } q,$$
$$p \vee q : p \text{ OR } q, \qquad p \to q : \text{If } p \text{ then } q,$$
$$p \leftrightarrow q : p \text{ if and only if } q$$

2. Truth Table Construction

A truth table lists all possible truth combinations of the propositions. For p and q:

p	q	$\neg p$	$p \wedge q$	$p \vee q$
T	T	F	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

3. Logical Equivalence

Two statements P and Q are logically equivalent if $P \leftrightarrow Q$ is a tautology. Examples:

$$(p \to q) \equiv (\neg p \vee q), \qquad \neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$$

4. Tautology, Contradiction and Contingency

- **Tautology:** Always true, e.g. $p \vee \neg p$.
- **Contradiction:** Always false, e.g. $p \wedge \neg p$.
- **Contingency:** True in some cases, false in others.

List of Experiments

1. Identify and classify simple and compound propositions.
2. Construct truth tables manually and through Python.
3. Verify logical equivalences using both manual and automated evaluation.
4. Determine tautologies and contradictions programmatically.

Logical and Coding Solution

Experiment 1: Classification of Propositions

- "The sky is blue." $\rightarrow$ Proposition (True)
- "Close the window." $\rightarrow$ Not a proposition (imperative)
- "$x + 3 = 5$." $\rightarrow$ Not a proposition (depends on x)

Experiment 2: Manual Verification

Verify that $(p \rightarrow q)$ and $(\neg p \vee q)$ are equivalent.

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Hence, both have identical truth values.

Experiment 3: Python Code for Truth Table Generation

Code:

```
import itertools

def truth_table(expr, vars):
  print(" | ".join(vars) + " | Result")
  for values in itertools.product([False, True], repeat=len(vars)):
    env = dict(zip(vars, values))
    res = eval(expr, , env)
    row = " | ".join(['T' if v else 'F' for v in values])
    print(row + " | " + ('T' if res else 'F'))

truth_table("(p and q) or (not p)", ["p","q"])
```

Experiment 4: Python Program for Equivalence Checking

```
def equivalent(expr1, expr2, vars):
  import itertools
  for values in itertools.product([False, True], repeat=len(vars)):
    env = dict(zip(vars, values))
    if eval(expr1, , env) != eval(expr2, , env):
      return False
  return True

print("Equivalent:", equivalent("(p and q) or (not p)",
"(not p) or q", ["p","q"]))
```

Discussion on Solution

Python's boolean operations (`and`, `or`, `not`) mirror logical connectives, enabling automatic generation and verification of truth tables. This helps visualize equivalences and tautologies efficiently, particularly for circuit validation or rule-based programming.

Sample Input and Output

Input:

```
truth_table("(p and q) or (not p)", ["p","q"])
```

Output:

```
p | q | Result
F | F | T
F | T | T
T | F | F
T | T | T
```

Input:

```
equivalent("(p and q) or (not p)", "(not p) or q", ["p","q"])
```

Output:

```
Equivalent:  True
```

Discussion and Inference

Logical reasoning can be verified computationally. The equivalence $(p \rightarrow q) \equiv (\neg p \vee q)$ confirms propositional consistency both symbolically and programmatically.

Exercises

1. Construct the truth table for $(p \wedge q) \rightarrow (p \vee q)$ using the Python script.
2. Write a program to check whether $(p \vee q) \wedge (\neg p \vee q)$ is equivalent to q.
3. Determine if $(p \vee \neg p)$ is a tautology programmatically.
4. Extend the script for three variables p, q, r and evaluate $(p \vee q) \rightarrow r$.
5. Explain how propositional logic underlies conditional statements in programming languages.

Lab 2: Predicates, Quantifiers, and Rules of Inference

Objective

To understand predicates, quantifiers, and basic rules of inference, and to verify logical statements involving universal and existential quantifiers using both manual reasoning and computational approaches in Python.

Theory

Predicate logic extends propositional logic by dealing with variables, functions, and quantifiers. It allows expressing statements such as "For all x" or "There exists an x" that cannot be represented in propositional logic.

1. Predicates and Propositions

A **predicate** is a statement containing one or more variables that becomes a proposition when specific values are substituted for the variables.
Example: $P(x) : x > 3$ — becomes a proposition when x is given a value, e.g., $P(5) \rightarrow$ True, $P(2) \rightarrow$ False.

2. Quantifiers

Quantifiers specify the scope of a predicate over a domain.
Universal Quantifier ($\forall$) — "For all":

$$\forall x\, P(x) \text{ is true if } P(x) \text{ holds for every } x \text{ in the domain.}$$

Existential Quantifier ($\exists$) — "There exists":

$$\exists x\, P(x) \text{ is true if } P(x) \text{ holds for at least one } x \text{ in the domain.}$$

Example: $\forall x(x^2 \geq 0)$ is true, and $\exists x(x^2 = 4)$ is also true.

3. Negation of Quantifiers

$$\neg(\forall x P(x)) \equiv \exists x(\neg P(x)), \qquad \neg(\exists x P(x)) \equiv \forall x(\neg P(x))$$

4. Rules of Inference

Rules of inference allow deriving conclusions from given premises logically.

Modus Ponens:	$(p \to q), p \vdash q$
Modus Tollens:	$(p \to q), \neg q \vdash \neg p$
Universal Instantiation:	$\forall x P(x) \vdash P(c)$
Existential Generalization:	$P(c) \vdash \exists x P(x)$

List of Experiments

1. Express English statements using predicate logic.
2. Determine truth values of quantified statements manually and in Python.
3. Verify the negation of quantified statements programmatically.
4. Demonstrate rules of inference using simple logical expressions.

Logical and Coding Solution

Experiment 1: Translating English Statements into Predicate Logic

- "All humans are mortal." $\to \forall x(Human(x) \to Mortal(x))$
- "Some students are programmers." $\to \exists x(Student(x) \land Programmer(x))$
- "Not all birds can fly." $\to \neg(\forall x(Bird(x) \to CanFly(x)))$

Experiment 2: Manual Evaluation of Quantified Statements

Let domain $D = \{1, 2, 3, 4, 5\}$, and $P(x) : x^2 \leq 16$

$$\forall x\, P(x) : \text{ True (all } x \in D \text{ satisfy } x^2 \leq 16)$$

$$\exists x(x^2 = 25) : \text{ True (since } x = 5)$$

Experiment 3: Python Evaluation of Quantifiers

```
def forall(domain, predicate):
return all(predicate(x) for x in domain)
def exists(domain, predicate):
return any(predicate(x) for x in domain)
domain = [1,2,3,4,5]
predicate1 = lambda x:  x**2 <= 16
predicate2 = lambda x:  x**2 == 25
print("For all x (x^2 <= 16):", forall(domain, predicate1))
print("There exists x (x^2 == 25):", exists(domain, predicate2))
```

Output:

```
For all x (x^2 <= 16):  True
There exists x (x^2 == 25):  True
```

Experiment 4: Negation of Quantified Statements in Python

```
def negate_forall(domain, predicate):
return any(not predicate(x) for x in domain)
def negate_exists(domain, predicate):
return all(not predicate(x) for x in domain)
print("Negation of forall x (x^2 <= 16):", negate_forall(domain, predicate1))
print("Negation of exists x (x^2 == 25):", negate_exists(domain, predicate2))
```

Output:

```
Negation of forall x (x^2 <= 16):  False
Negation of exists x (x^2 == 25):  False
```

Experiment 5: Demonstration of Modus Ponens in Python

```
def modus_ponens(p, q):
return (p and (not p or q)) # p -> q equivalent to not p or q

   p = True
q = True
print("Modus Ponens Result:", modus_ponens(p, q))
```

Output:

```
Modus Ponens Result:  True
```

Discussion on Solution

Predicate logic enhances the expressiveness of propositional logic, enabling formal representation of general mathematical and real-world statements. By implementing quantifiers computationally, one can simulate universal and existential validation for specific domains. The rules of inference implemented in Python mirror the logical flow used in automated theorem proving and AI reasoning systems.

Sample Input and Output

Input:

```
forall([1,2,3,4,5], lambda x:  x**2 <= 16)
exists([1,2,3,4,5], lambda x:  x**2 == 25)
```

Output:

```
True
True
```

Input:

```
negate_forall([1,2,3,4,5], lambda x:  x**2 <= 16)
negate_exists([1,2,3,4,5], lambda x:  x**2 == 25)
```

Output:

```
False
False
```

Exercises

1. Translate and evaluate the following using Python:
   (a) "All even numbers are divisible by 2."
   (b) "There exists a number divisible by 3 and 5."
2. Write a Python function that returns `True` if a given domain satisfies both $\forall x P(x)$ and $\exists x Q(x)$.
3. Implement a program to verify $\neg(\forall x P(x)) \equiv \exists x(\neg P(x))$.
4. Extend the domain to strings and test a predicate $P(s)$: "s starts with a vowel."
5. Explain how rules of inference such as Modus Ponens and Modus Tollens can be automated in reasoning systems.

Lab 3: Set Operations and Computer Representation of Sets

Objective

To understand fundamental concepts of set theory, perform set operations such as union, intersection, and complement, and explore how sets can be represented and manipulated computationally using Python.

Theory

A **set** is a collection of distinct objects, considered as an object in its own right. In computer science, sets represent collections of unique elements — such as database keys, graph vertices, or symbols in automata.

1. Basic Set Definitions

If $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$:

$$\begin{aligned} A \cup B &= \{1, 2, 3, 4, 5\} && \text{(Union)} \\ A \cap B &= \{3\} && \text{(Intersection)} \\ A - B &= \{1, 2\} && \text{(Difference)} \\ A \oplus B &= \{1, 2, 4, 5\} && \text{(Symmetric Difference)} \end{aligned}$$

2. Complement of a Set

Given a universal set U, the complement of A is:

$$A' = U - A$$

If $U = \{1, 2, 3, 4, 5, 6\}$ and $A = \{2, 4, 6\}$, then $A' = \{1, 3, 5\}$.

3. Cartesian Product

The Cartesian product of two sets A and B is defined as:

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

If $A = \{1, 2\}$, $B = \{x, y\}$, then $A \times B = \{(1, x), (1, y), (2, x), (2, y)\}$.

4. Representation of Sets in Computer Memory

In programming, sets are often represented using:

- **Bit Strings:** Each bit represents membership (1 for present, 0 for absent).
- **Built-in Data Structures:** Python's `set()` type implements hash-based sets for efficient operations.

List of Experiments

1. Perform basic set operations (union, intersection, difference, symmetric difference).
2. Compute the complement of a set with respect to a universal set.
3. Generate Cartesian products of sets.
4. Represent sets as bit strings and verify equivalence with Python's `set` operations.

Logical and Coding Solution

Experiment 1: Set Operations in Python

```
A = {1, 2, 3}
B = {3, 4, 5}
U = {1, 2, 3, 4, 5, 6}
print("A union B =", A.union(B))
print("A intersection B =", A.intersection(B))
print("A - B =", A.difference(B))
print("A symmetric_difference B =", A.symmetric_difference(B))
print("Complement of A =", U.difference(A))
```

Output:

```
A union B = {1, 2, 3, 4, 5}
A intersection B = {3}
A - B = {1, 2}
A symmetric_difference B = {1, 2, 4, 5}
Complement of A = {4, 5, 6}
```

Experiment 2: Cartesian Product

```
A = {1, 2}
B = {'x', 'y'}
cartesian = {(a, b) for a in A for b in B}
print("A × B =", cartesian)
```

Output:

```
A × B = {(1, 'x'), (1, 'y'), (2, 'x'), (2, 'y')}
```

Experiment 3: Bit String Representation of Sets

```
def set_to_bitstring(U, S):
return [1 if x in S else 0 for x in U]
def bitstring_to_set(U, bits):
return {U[i] for i, bit in enumerate(bits) if bit == 1}
U = [1, 2, 3, 4, 5, 6]
A = {2, 4, 6}
bits = set_to_bitstring(U, A)
print("Bitstring for A:", bits)
print("Reconstructed Set:", bitstring_to_set(U, bits))
```

Output:

```
Bitstring for A: [0, 1, 0, 1, 0, 1]
Reconstructed Set:  {2, 4, 6}
```

Experiment 4: Verification of Set Identities

Example: Verify the identity $(A \cup B)' = A' \cap B'$.

```
U = {1, 2, 3, 4, 5, 6}
A = {1, 2, 3}
B = {3, 4, 5}
lhs = U.difference(A.union(B))
rhs = U.difference(A).intersection(U.difference(B))
print("LHS =", lhs)
print("RHS =", rhs)
print("Identity Verified:", lhs == rhs)
```

Output:

```
LHS = {6}
RHS = {6}
Identity Verified:  True
```

Discussion on Solution

Python's built-in `set` type efficiently supports mathematical set operations. Bitstring representation illustrates how computers internally manage sets for Boolean logic, data compression, and bitmask operations. Identity verification demonstrates logical equivalence between symbolic and computational set expressions.

Sample Input and Output

Input:

```
A = {1, 2, 3}
B = {3, 4, 5}
U = {1, 2, 3, 4, 5, 6}
```

Output:

```
A union B = {1, 2, 3, 4, 5}
A intersection B = {3}
```

```
A complement = {4, 5, 6}
(A union B) complement = {6} Identity Verified:  True
```

Discussion and Inference

This lab bridges theoretical set algebra with practical computation. By verifying symbolic laws through code, students reinforce understanding of set properties and logical equivalence — essential for database design, automata theory, and digital logic circuits.

Exercises

1. Verify the following identities using Python:
   (a) $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
   (b) $(A \cup B)' = A' \cap B'$
2. Represent $A = \{2, 4, 6, 8\}$ as a bitstring when $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$.
3. Write a function to compute $A \times B$ for arbitrary sets.
4. Write a program to check if two sets are disjoint.
5. Explain one real-world application of set operations in computer science.

Lab 4: Functions and Mappings (One-to-One, Onto, and Composition)

Objective

To understand the properties of mathematical functions and mappings, including one-to-one (injective), onto (surjective), and bijective functions, and to implement function composition and property verification computationally using Python.

Theory

A **function** is a special type of relation where each element of a set A (domain) is associated with exactly one element of another set B (codomain). We denote it as $f : A \to B$.

1. Definitions

- **Injective (One-to-One):** A function is injective if different inputs map to different outputs. $f(x_1) = f(x_2) \Rightarrow x_1 = x_2$
- **Surjective (Onto):** A function $f : A \to B$ is surjective if every element of B has at least one preimage in A.
- **Bijective:** A function that is both injective and surjective.
- **Composition of Functions:** If $f : A \to B$ and $g : B \to C$, then $(g \circ f)(x) = g(f(x))$.

2. Example

Let $f(x) = 2x + 1$ and $g(x) = x^2$. Then $(g \circ f)(x) = (2x + 1)^2$.

List of Experiments

1. Identify whether a given mapping is a function.
2. Test if a function is injective, surjective, or bijective.

3. Compute the composition of two functions.
4. Visualize mapping relations using domain and range pairs.

Logical and Coding Solution

Experiment 1: Determine if a Relation is a Function

```
def is_function(pairs):
domain = [x for x, _ in pairs]
return len(domain) == len(set(domain))
relation1 = [(1,2), (2,3), (3,4)]
relation2 = [(1,2), (1,3), (2,4)]
print("R1 is function:", is_function(relation1))
print("R2 is function:", is_function(relation2))
```

Output:

```
R1 is function:  True
R2 is function:  False
```

Experiment 2: Check Injective and Surjective Properties

```
def is_injective(pairs):
images = [y for _, y in pairs]
return len(images) == len(set(images))
def is_surjective(pairs, codomain):
images = {y for _, y in pairs}
return images == set(codomain)
A = [1,2,3]
B = [2,3,4]
relation = [(1,2), (2,3), (3,4)]
print("Injective:", is_injective(relation))
print("Surjective:", is_surjective(relation, B))
```

Output:

```
Injective:  True
Surjective:  True
```

Experiment 3: Composition of Functions

```
def f(x):  return 2*x + 1
def g(x):  return x^2
def composition(f, g, domain):
return [(x, g(f(x))) for x in domain]
domain = [1,2,3,4]
print("Composition g(f(x)):", composition(f, g, domain))
```

Output:

```
Composition g(f(x)):  [(1,9), (2,25), (3,49), (4,81)]
```

Experiment 4: Mapping Visualization (Optional)

```
import matplotlib.pyplot as plt
A = [1,2,3]
B = [2,3,4]
mapping = dict(zip(A,B))
plt.scatter(A, [1]*len(A), color='blue', label='Domain')
plt.scatter(B, [2]*len(B), color='red', label='Codomain')
for a,b in mapping.items():
plt.plot([a,b],[1,2],'k-')
plt.yticks([1,2], ['Domain (A)','Codomain (B)'])
plt.legend()
plt.title("Visual Representation of Mapping f:  A to B")
plt.show()
```

Output:
A mapping diagram showing connections between domain and codomain elements.

Discussion on Solution

This lab demonstrates how mathematical mappings translate naturally into computational relationships. Injectivity corresponds to uniqueness in output mapping, surjectivity ensures full coverage of the codomain, and composition models sequential function calls — a common structure in algorithms and data pipelines.

Sample Input and Output

Input:

```
relation = [(1,2), (2,3), (3,4)]
codomain = [2,3,4]
```

Output:

```
R is function:  True
Injective:  True
Surjective:  True
Composition g(f(x)):  [(1,9), (2,25), (3,49)]
```

Discussion and Inference

Every function can be analyzed algorithmically for injectivity and surjectivity. Compositions demonstrate how multiple transformations interact — an essential concept in mathematics, programming, and machine learning pipelines.

Exercises

1. For $f(x) = 3x+1$ and $g(x) = x-2$, compute $(g \circ f)(x)$ and $(f \circ g)(x)$ using Python.

2. Write a program that checks whether a given mapping from set A to set B is bijective.

3. Create a visual representation for a non-injective function and analyze its behavior.

4. Verify if $f(x) = x^2$ is injective and surjective over domain $[-2, 2]$ and codomain $[0, 4]$.

5. Explain one real-world scenario where function composition is useful, such as in data transformation or machine learning pipelines.

Lab 5: Sequences, Recurrence Relations, and Summations

Objective

To understand the concepts of sequences, recurrence relations, and summations, and to implement their computation and visualization using Python programming.

Theory

A **sequence** is an ordered list of numbers that follows a specific rule. A **recurrence relation** defines each term of a sequence as a function of its preceding terms. **Summations** represent the addition of terms in a sequence.

1. Sequences

A sequence can be written as $\{a_n\}$, where a_n is the n^{th} term.
Example forms: Arithmetic Sequence: $a_n = a_1 + (n-1)d$ Geometric Sequence: $a_n = a_1 r^{n-1}$

2. Recurrence Relations

A recurrence relation expresses a_n in terms of earlier terms. Example: $a_n = a_{n-1} + a_{n-2},\ a_0 = 0,\ a_1 = 1$ defines the Fibonacci sequence.

3. Summations

Summation represents the total of sequence terms: $\sum_{i=1}^{n} a_i = a_1 + a_2 + \ldots + a_n$ Example: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$

4. Solving Recurrences

Some recurrences can be solved explicitly using iteration or characteristic equations. Example: $a_n = 2a_{n-1},\ a_0 = 1$ gives $a_n = 2^n$.

List of Experiments

1. Generate arithmetic and geometric sequences.
2. Implement recursive and iterative Fibonacci sequences.
3. Evaluate summations manually and programmatically.
4. Solve recurrence relations iteratively.

Logical and Coding Solution

Experiment 1: Arithmetic and Geometric Sequences

```
def arithmetic_sequence(a1, d, n):
return [a1 + i*d for i in range(n)]
def geometric_sequence(a1, r, n):
return [a1 * (r**i) for i in range(n)]
print("Arithmetic Sequence:", arithmetic_sequence(2,3,5))
print("Geometric Sequence:", geometric_sequence(2,2,5))
```

Output:

```
Arithmetic Sequence:  [2, 5, 8, 11, 14]
Geometric Sequence:  [2, 4, 8, 16, 32]
```

Experiment 2: Fibonacci Sequence (Recursive and Iterative Methods)

Recursive Implementation:

```
def fib_recursive(n):
if n <= 1:  return n
return fib_recursive(n-1) + fib_recursive(n-2)
print("Fibonacci (recursive):", [fib_recursive(i) for i in range(10)])
```

Iterative Implementation:

```
def fib_iterative(n):
seq = [0,1]
for i in range(2, n):  seq.append(seq[-1] + seq[-2])
return seq[:n]
print("Fibonacci (iterative):", fib_iterative(10))
```

Output:

```
Fibonacci (recursive):  [0,1,1,2,3,5,8,13,21,34]
Fibonacci (iterative):  [0,1,1,2,3,5,8,13,21,34]
```

Experiment 3: Summation of a Sequence

```
def summation(seq):  return sum(seq)
A = arithmetic_sequence(2,3,5)
print("Sum of Arithmetic Sequence:", summation(A))
```

Output:

```
Sum of Arithmetic Sequence:  40
```

Experiment 4: Solve Recurrence Using Iteration

```
def recurrence(a0, n):
seq = [a0]
for i in range(1, n):  seq.append(2 * seq[-1])
return seq
print("Recurrence Sequence:", recurrence(1,6))
```

Output:

```
Recurrence Sequence:  [1, 2, 4, 8, 16, 32]
```

Experiment 5: Visualization of Sequences

```
import matplotlib.pyplot as plt
n = range(10)
fib = fib_iterative(10)
plt.plot(n, fib, marker='o')
plt.title("Fibonacci Sequence Visualization")
plt.xlabel("n")
plt.ylabel("Fibonacci(n)")
plt.grid(True)
plt.show()
```

Output:

Graph displaying Fibonacci numbers versus indices.

Discussion on Solution

Arithmetic and geometric sequences model uniform and exponential growth. Recurrence relations capture recursive processes fundamental to dynamic programming. Summations represent cumulative computations essential in analysis and algorithm design. Visualizations help identify growth patterns and relationships between successive terms.

Sample Input and Output

Input:

```
a1, d, n = 2, 3, 5
arithmetic_sequence(a1, d, n)
fib_iterative(10)
```

Output:

```
Arithmetic Sequence:  [2, 5, 8, 11, 14]
Fibonacci:  [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Discussion and Inference

Sequences and recurrence relations connect mathematical abstraction to algorithmic logic. Through coding, learners explore iterative, recursive, and summation structures critical in computational mathematics and data science.

Exercises

1. Derive a formula for the n^{th} term and sum of a geometric progression using code verification.
2. Write a function to compute $a_n = 3a_{n-1} - 2$ with $a_0 = 1$.
3. Modify the Fibonacci function to compute Lucas numbers.
4. Plot both arithmetic and geometric sequences for $a_1 = 2, d = 3, r = 2$.
5. Discuss the computational complexity of recursive versus iterative recurrence solving.

Lab 6: Counting Principles and Combinatorics

Objective

To understand the fundamental principles of counting, including factorial, permutation, and combination, and to implement and verify these principles using Python programming.

Theory

Combinatorics deals with counting, arranging, and selecting objects in structured ways. The principles of counting form the basis of probability, graph theory, and algorithmic complexity analysis.

1. Fundamental Counting Principle

If a process consists of k independent steps where each step can be performed in n_i ways, then the total number of possible outcomes is:

$$N = n_1 \times n_2 \times \cdots \times n_k$$

Example: choosing 2 shirts and 3 pants gives $2 \times 3 = 6$ combinations.

2. Factorial

The factorial of a positive integer n is defined as $n! = n \times (n-1) \times (n-2) \times \cdots \times 1$. Example: $5! = 120$

3. Permutation

A **permutation** represents an arrangement of objects. The number of permutations of r objects selected from n is given by $P(n,r) = \frac{n!}{(n-r)!}$. Example: $P(5,2) = 5 \times 4 = 20$.

4. Combination

A **combination** represents selection without regard to order. The number of combinations is $C(n,r) = \frac{n!}{r!(n-r)!}$. Example: $C(5,2) = 10$.

5. Relationship Between Permutation and Combination

$P(n, r) = C(n, r) \times r!$

List of Experiments

1. Implement factorial computation (recursive and iterative).
2. Compute permutations and combinations for given n and r.
3. Verify the relation $P(n, r) = C(n, r) \times r!$.
4. Generate all possible permutations and combinations for a set.
5. Visualize combinatorial growth using Python plotting.

Logical and Coding Solution

Experiment 1: Factorial Function

Recursive Implementation:

```
def factorial_recursive(n):
if n == 0 or n == 1:  return 1
return n * factorial_recursive(n - 1)
print("5!  =", factorial_recursive(5))
```

Iterative Implementation:

```
def factorial_iterative(n):
result = 1
for i in range(2, n + 1):  result *= i
return result
print("6!  =", factorial_iterative(6))
```

Output:

```
5!  = 120
6!  = 720
```

Experiment 2: Permutations and Combinations

```
def permutation(n, r):
return factorial_iterative(n) // factorial_iterative(n - r)
def combination(n, r):
return factorial_iterative(n) // (factorial_iterative(r) * factorial_iterative(n
- r))
n, r = 5, 2
print("P(5, 2) =", permutation(n, r))
print("C(5, 2) =", combination(n, r))
```

Output:

```
P(5, 2) = 20
C(5, 2) = 10
```

Experiment 3: Verification of P(n,r) = C(n,r) × r!

```
n, r = 6, 3
lhs = permutation(n, r)
rhs = combination(n, r) * factorial_iterative(r)
print("Relation holds:", lhs == rhs)
```

Output:

```
Relation holds:  True
```

Experiment 4: Generate All Permutations and Combinations

```
import itertools
data = ['A', 'B', 'C']
print("Permutations of length 2:")
for p in itertools.permutations(data, 2):  print(p)
print("\nCombinations of length 2:")
for c in itertools.combinations(data, 2):  print(c)
```

Output:

```
Permutations of length 2:
('A','B')
('A','C')
('B','A')
('B','C')
('C','A')
('C','B')
Combinations of length 2:
('A','B')
('A','C')
('B','C')
```

Experiment 5: Visualization of Factorial Growth

```
import matplotlib.pyplot as plt
n_values = range(1, 8)
factorials = [factorial_iterative(n) for n in n_values]
plt.plot(n_values, factorials, marker='o')
plt.title("Factorial Growth Curve")
plt.xlabel("n")
plt.ylabel("n!")
plt.grid(True)
plt.show()
```

Output:

A plot showing the exponential growth of factorial values as n increases.

Discussion on Solution

This lab demonstrates the rapid growth behavior of factorial and combinatorial functions. Python enables direct verification of relationships among factorial, permutation, and

combination. Recursive functions highlight the mathematical definition of factorials, while `itertools` simplifies combinatorial generation for algorithmic exploration.

Sample Input and Output

Input:

```
n = 5; r = 2
print(permutation(n, r))
print(combination(n, r))
```

Output:

```
20
10
```

Discussion and Inference

Factorial and combinatorial principles are fundamental for understanding algorithmic complexity ($O(n!)$ growth), probability analysis, and discrete modeling. Implementing these using Python strengthens comprehension of mathematical reasoning and computational efficiency.

Exercises

1. Write a Python function to compute $C(n, r)$ recursively.
2. Generate all 3-letter passwords from the set $\{A, B, C, D\}$ without repetition.
3. Verify the identity $C(n, r) = C(n, n - r)$ for multiple values of n and r.
4. Plot both $P(n, r)$ and $C(n, r)$ for $n = 1$ to 8 with $r = 3$.
5. Discuss one real-world application of combinatorics in computer science.

Lab 7: Number Theory and Modular Arithmetic

Objective

To understand the concepts of divisibility, greatest common divisor (GCD), least common multiple (LCM), and modular arithmetic, and to verify number-theoretic properties computationally using Python.

Theory

Number theory deals with properties of integers and their relationships. Modular arithmetic introduces cyclic behavior of numbers, foundational for computer cryptography and coding theory.

1. Divisibility

An integer a divides b (denoted $a|b$) if there exists an integer k such that $b = a \times k$. Example: $3|12$ since $12 = 3 \times 4$.

2. GCD and LCM

Greatest Common Divisor (GCD): The largest integer dividing both a and b. $\gcd(a, b) =$ largest integer such that $a \bmod g = 0$, $b \bmod g = 0$.
Least Common Multiple (LCM): $\text{lcm}(a, b) = \frac{|a \times b|}{\gcd(a,b)}$

3. Euclidean Algorithm for GCD

The Euclidean Algorithm efficiently finds the GCD:

$$\gcd(a, b) = \begin{cases} a, & \text{if } b = 0 \\ \gcd(b, a \bmod b), & \text{otherwise} \end{cases}$$

4. Modular Arithmetic

For integers a, b, n: $a \equiv b \pmod{n}$ if $n|(a - b)$. Example: $17 \equiv 5 \pmod{12}$ because $17 - 5 = 12$.

5. Modular Inverse

An integer a^{-1} is the modular inverse of a modulo n if $a \times a^{-1} \equiv 1 \pmod{n}$. It exists only when $\gcd(a, n) = 1$.

List of Experiments

1. Check divisibility and compute GCD and LCM.
2. Implement Euclidean and Extended Euclidean algorithms.
3. Perform modular arithmetic operations in Python.
4. Compute modular inverse using the Extended Euclidean algorithm.
5. Verify modular congruences and properties.

Logical and Coding Solution

Experiment 1: GCD and LCM Computation

```
import math
a, b = 36, 60
print("GCD using math.gcd:", math.gcd(a, b))
def lcm(a, b):  return abs(a*b) // math.gcd(a, b)
print("LCM:", lcm(a, b))
```

Output:

```
GCD using math.gcd:  12
LCM: 180
```

Experiment 2: Euclidean Algorithm

```
def gcd_euclid(a, b):
while b:  a, b = b, a % b
return a
print("GCD(48,18) =", gcd_euclid(48,18))
```

Output:

```
GCD(48,18) = 6
```

Experiment 3: Extended Euclidean Algorithm (Modular Inverse)

```
def extended_gcd(a, b):
if b == 0:  return a, 1, 0
g, x1, y1 = extended_gcd(b, a % b)
x = y1
y = x1 - (a // b) * y1
return g, x, y
```

```
def mod_inverse(a, n):
g, x, y = extended_gcd(a, n)
if g != 1:  raise ValueError("Inverse does not exist.")
else:  return x % n
a, n = 7, 26
print("Modular Inverse of 7 mod 26 =", mod_inverse(a, n))
```

Output:

```
Modular Inverse of 7 mod 26 = 15
```

Experiment 4: Modular Arithmetic Operations

```
a, b, n = 17, 5, 12
print("(a + b) mod n =", (a + b) % n)
print("(a * b) mod n =", (a * b) % n)
print("(a - b) mod n =", (a - b) % n)
```

Output:

```
(a + b) mod n = 10
(a * b) mod n = 1
(a - b) mod n = 0
```

Experiment 5: Verification of Congruence Property

```
def check_congruence(a, b, n):  return (a - b) % n == 0
print("17 \equiv 5 (mod 12)?", check_congruence(17, 5, 12))
```

Output:

```
17 \equiv 5 (mod 12)?  True
```

Discussion on Solution

This lab demonstrates algorithmic reasoning within number theory. The Euclidean algorithm illustrates recursive reduction, while modular arithmetic underpins hashing, encryption, and digital system design. Python's libraries simplify integer computations, while algorithmic coding enhances mathematical comprehension.

Sample Input and Output

Input:

```
a, b = 48, 18
gcd_euclid(a, b)
mod_inverse(7, 26)
```

Output:

```
GCD(48, 18) = 6
Modular Inverse of 7 mod 26 = 15
```

Discussion and Inference

Number theory provides the computational foundation of secure communication and algorithm design. Concepts such as modular inverses and congruences are central to RSA encryption, hashing, and coding theory. Programming these techniques helps bridge mathematical proofs and practical implementations.

Exercises

1. Write a Python function to compute the GCD of three numbers.
2. Verify that $(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$.
3. Implement modular exponentiation: compute $a^b \bmod n$ efficiently.
4. Find the modular inverse of 11 modulo 31 and verify it.
5. Discuss one real-world application of modular arithmetic in cryptography.

Lab 8: Solving Linear Congruences and Modular Equations

Objective

To learn how to solve linear congruences of the form $ax \equiv b \pmod{n}$ using mathematical reasoning and the Extended Euclidean Algorithm, and to verify solutions computationally using Python.

Theory

A **linear congruence** is an equation of the form $ax \equiv b \pmod{n}$, where $a, b, n \in \mathbb{Z}$ and $n > 0$. It seeks integer values of x such that $a \times x$ divided by n leaves remainder b.

1. Existence of Solution

The congruence $ax \equiv b \pmod{n}$ has a solution if and only if $\gcd(a, n)$ divides b.

2. Number of Solutions

If $d = \gcd(a, n)$ divides b, then there are exactly d distinct solutions modulo n.

3. General Solution

If $ax \equiv b \pmod{n}$ and $d = \gcd(a, n)$, then $x = x_0 + k \times \frac{n}{d}$, $k = 0, 1, 2, \ldots, d-1$, where x_0 is a particular solution obtained via the Extended Euclidean Algorithm.

4. Modular Inverse Method

If $\gcd(a, n) = 1$, then a unique solution exists: $x \equiv b \times a^{-1} \pmod{n}$, where a^{-1} is the modular inverse of a modulo n.

List of Experiments

1. Verify existence of solutions for given a, b, n.
2. Solve $ax \equiv b \pmod{n}$ using the Extended Euclidean Algorithm.
3. Compute all possible solutions when $\gcd(a, n) > 1$.

4. Implement the modular inverse method for solvable congruences.
5. Test congruence solutions programmatically for verification.

Logical and Coding Solution

Experiment 1: Check Existence of Solution

```
import math
def has_solution(a, b, n):  return b % math.gcd(a, n) == 0
print("Does 14x \equiv 30 (mod 100) have a solution?", has_solution(14, 30,
100))
```

Output:

```
Does 14x \equiv 30 (mod 100) have a solution?  True
```

Experiment 2: Extended Euclidean Algorithm (for Solving Congruence)

```
def extended_gcd(a, b):
if b == 0:  return a, 1, 0
g, x1, y1 = extended_gcd(b, a % b)
x = y1
y = x1 - (a // b) * y1
return g, x, y
def solve_congruence(a, b, n):
g, x, y = extended_gcd(a, n)
if b % g != 0:  return None
x0 = (x * (b // g)) % n
return [(x0 + i * (n // g)) % n for i in range(g)]
print("Does 14x \equiv 30 (mod 100) have a solution?", has_solution(14, 30,
100))
```

Output:

```
Does 14x \equiv 30 (mod 100) have a solution?  True
```

Experiment 3: Modular Inverse Method (When gcd(a,n)=1)

```
def mod_inverse(a, n):
g, x, y = extended_gcd(a, n)
if g != 1:  raise ValueError("Inverse does not exist")
return x % n
a, b, n = 7, 5, 26
inv = mod_inverse(a, n)
x = (b * inv) % n
print("Solution of 7x \equiv 5 (mod 26):  x =", x)
```

Output:

```
Solution of 7x \equiv 5 (mod 26):  x = 23
```

Experiment 4: Verification of Congruence Solution

```
def verify_congruence(a, x, b, n):  return (a * x - b) % n == 0
print("Verification:", verify_congruence(7, 23, 5, 26))
```

Output:

```
Verification:  True
```

Experiment 5: Visualization of Modular Residues (Optional)

```
import matplotlib.pyplot as plt
n = 10
residues = [a % n for a in range(20)]
plt.stem(range(20), residues)
plt.title("Residue Classes modulo 10")
plt.xlabel("a")
plt.ylabel("a mod 10")
plt.show()
```

Output:

A stem plot displaying the repeating residue pattern modulo 10.

Discussion on Solution

This lab connects modular arithmetic with computational number theory. The Extended Euclidean Algorithm not only determines the GCD but also provides modular inverses, enabling congruence solutions. Python implementation confirms theoretical results and residue visualization shows periodic behavior in modular systems.

Sample Input and Output

Input:

```
solve_congruence(14, 30, 100)
solve_congruence(7, 5, 26)
```

Output:

```
[95, 45]
[23]
```

Discussion and Inference

Linear congruences are key to cryptography, hashing, and modular algorithms. By computing modular inverses and verifying congruences programmatically, one gains deeper insight into arithmetic systems that underpin encryption and number-theoretic computations.

Exercises

1. Solve $12x \equiv 18 \pmod{30}$ manually and verify using Python.

2. Implement a program that lists all x satisfying $9x \equiv 15 \pmod{24}$.
3. Write a Python function to determine if a congruence has a unique solution.
4. Plot residues for modulo $n = 7$ and $n = 12$ to observe periodicity.
5. Explain the importance of solving modular equations in cryptography (e.g., RSA algorithm).

Lab 9: Mathematical Induction and Recursion

Objective

To understand and apply the principles of mathematical induction for proving statements about integers, and to explore the relationship between induction and recursion through theoretical and computational approaches.

Theory

Mathematical Induction is a proof technique used to establish that a statement holds for all natural numbers. It involves two main steps:

1. **Base Case:** Verify that the statement is true for the initial value (usually $n = 1$).
2. **Inductive Step:** Assume that the statement holds for $n = k$, and then prove it for $n = k + 1$.

If both steps hold, the statement is true for all $n \in \mathbb{N}$.

1. Example of Inductive Proof

Prove that $1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$.
Base Case: For $n = 1$, $1 = \frac{1(1+1)}{2} = 1$. True.
Inductive Step: Assume true for $n = k$: $1 + 2 + \ldots + k = \frac{k(k+1)}{2}$. Then for $n = k + 1$:

$$1 + 2 + \ldots + k + (k + 1) = \frac{k(k + 1)}{2} + (k + 1)$$

$$= (k + 1)\left(\frac{k}{2} + 1\right) = \frac{(k + 1)(k + 2)}{2}$$

Hence proved for $n = k + 1$.

2. Recursion

Recursion is the computational counterpart of induction, where a function is defined in terms of itself. Example:

$$f(n) = \begin{cases} 1, & n = 0 \\ n + f(n-1), & n > 0 \end{cases}$$

3. Connection Between Induction and Recursion

Induction proves correctness, while recursion performs computation. Each recursive step parallels the inductive step logically.

List of Experiments

1. Prove simple summation formulas using induction.
2. Implement recursive algorithms corresponding to inductive proofs.
3. Compare recursive and iterative solutions for the same problem.
4. Trace recursion using Python for factorial and summation.

Logical and Coding Solution

Experiment 1: Inductive Proof Verification in Python

Example: Verify $1 + 2 + 3 + \ldots + n = n(n+1)/2$.

```
def verify_sum_formula(n):
lhs = sum(range(1, n + 1))
rhs = n * (n + 1) // 2
return lhs == rhs
for n in range(1, 8):
print("n=", n, "Verified:", verify_sum_formula(n))
```

Output:

```
n=1, Verified:  True
n=2, Verified:  True
n=3, Verified:  True
n=4, Verified:  True
...
```

Experiment 2: Recursive Factorial Function

```
def factorial(n):
if n == 0:  return 1
return n * factorial(n - 1)
print("5!  =", factorial(5))
```

Output:

```
5!  = 120
```

Experiment 3: Recursive Summation Function

```
def recursive_sum(n):
if n == 0:  return 0
return n + recursive_sum(n - 1)
print("Sum of first 10 numbers:", recursive_sum(10))
```

Output:

```
Sum of first 10 numbers:  55
```

Experiment 4: Iterative vs Recursive Comparison

```
def iterative_sum(n):
total = 0
for i in range(1, n + 1):  total += i
return total
for n in [5, 10, 20]:
print("n=", n, ":  Recursive=", recursive_sum(n), ", Iterative=", iterative_sum(n))
```

Output:

```
n=5:  Recursive=15, Iterative=15
n=10:  Recursive=55, Iterative=55
n=20:  Recursive=210, Iterative=210
```

Experiment 5: Trace Recursion Calls (Visualization)

```
def trace_factorial(n, depth=0):
print(" " * depth + f"factorial(n) called")
if n == 0:
print(" " * depth + "returns 1")
return 1
result = n * trace_factorial(n - 1, depth + 1)
print(" " * depth + f"returns result")
return result
trace_factorial(4)
```

Output:

```
factorial(4) called
factorial(3) called
factorial(2) called
factorial(1) called
factorial(0) called
returns 1
```

```
returns 1
returns 2
returns 6
returns 24
```

Discussion on Solution

Mathematical induction provides the logical foundation for recursion. Each recursive function includes a base case (analogous to the inductive base) and a recursive call (analogous to the inductive step). Python's recursion elegantly models mathematical structures, though excessive recursion depth can lead to stack overflow.

Sample Input and Output

Input:

```
verify_sum_formula(10)
factorial(5)
recursive_sum(10)
```

Output:

```
True
120
55
```

Discussion and Inference

This lab bridges mathematical logic and computation. Induction proves correctness, while recursion enables implementation. Together, they illustrate the structural foundation of algorithms, allowing students to visualize how proofs translate into executable logic.

Exercises

1. Prove using induction: $1^2 + 2^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$.
2. Write a recursive function to compute $n^2 + (n-1)^2 + \ldots + 1^2$.
3. Compare recursive and iterative execution times for factorial.
4. Modify `trace_factorial` to display recursion depth dynamically.
5. Discuss how recursion relates to stack memory and base case termination.

Lab 10: Graph Models and Representation

Objective

To understand the basic concepts of graph theory and learn how to represent graphs using adjacency matrices and adjacency lists, both manually and through Python implementation.

Theory

A **graph** $G = (V, E)$ consists of two components:

$$V = \text{set of vertices (or nodes)}, \quad E = \text{set of edges connecting pairs of vertices.}$$

Graphs are widely used to model networks, relationships, and paths in computer science, such as in social networks, routing, and circuit design.

1. Types of Graphs

- **Undirected Graph:** Edges have no direction; $(u, v) = (v, u)$.
- **Directed Graph (Digraph):** Edges have direction; $(u, v) \neq (v, u)$.
- **Weighted Graph:** Each edge carries a numerical weight.
- **Simple Graph:** No self-loops or multiple edges between nodes.

2. Graph Representation

Graphs can be represented in multiple forms:

- **Adjacency Matrix:** A 2D matrix A where $A[i][j] = 1$ if there exists an edge between vertex i and j, otherwise 0.
- **Adjacency List:** Each vertex stores a list of its adjacent vertices.

3. Example

For $V = \{A, B, C\}$ and $E = \{(A, B), (B, C)\}$:

$$\text{Adjacency Matrix: } \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\text{Adjacency List: } A \to B \quad B \to A, C \quad C \to B$$

List of Experiments

1. Construct undirected and directed graphs manually.
2. Implement adjacency matrix and adjacency list in Python.
3. Create a weighted graph using Python dictionaries.
4. Visualize graphs using the `networkx` library.
5. Compare adjacency matrix and list in terms of space and performance.

Logical and Coding Solution

Experiment 1: Adjacency Matrix Representation

```
vertices = ['A', 'B', 'C']
edges = [('A', 'B'), ('B', 'C')]
n = len(vertices)
matrix = [[0] * n for _ in range(n)]
for u, v in edges:
i, j = vertices.index(u), vertices.index(v)
matrix[i][j] = matrix[j][i] = 1
print("Adjacency Matrix:")
for row in matrix:  print(row)
```

Output:

```
Adjacency Matrix:
[0, 1, 0]
[1, 0, 1]
[0, 1, 0]
```

Experiment 2: Adjacency List Representation

```
graph = {v:  [] for v in vertices}
for u, v in edges:
graph[u].append(v)
graph[v].append(u)
print("Adjacency List:")
for node, adj in graph.items():  print(node, "->", adj)
```

Output:

```
Adjacency List:
A -> ['B']
B -> ['A', 'C']
C -> ['B']
```

Experiment 3: Weighted Graph Representation

```
weighted_edges = [('A', 'B', 4), ('B', 'C', 2), ('A', 'C', 5)]
weighted_graph = {}
for u, v, w in weighted_edges:
weighted_graph.setdefault(u, []).append((v, w))
weighted_graph.setdefault(v, []).append((u, w))
print("Weighted Graph:")
for node, adj in weighted_graph.items():  print(node, "->", adj)
```

Output:

```
Weighted Graph:
A -> [('B', 4), ('C', 5)]
B -> [('A', 4), ('C', 2)]
C -> [('B', 2), ('A', 5)]
```

Experiment 4: Graph Visualization Using NetworkX

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
G.add_weighted_edges_from(weighted_edges)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color='lightblue',
node_size=1200, font_size=12)
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.title("Weighted Undirected Graph")
plt.show()
```

Output:

A visual graph showing nodes A, B, and C connected by weighted edges 4, 2, and 5.

Experiment 5: Space Comparison Between Representations

```
def matrix_space(n):  return n * n
def list_space(graph):  return sum(len(adj) for adj in graph.values())
```

```
print("Matrix Space:", matrix_space(len(vertices)))
print("List Space:", list_space(graph))
```
Output:
```
Matrix Space:  9
List Space:  4
```

Discussion on Solution

This lab demonstrates how graphs can be represented and manipulated computationally. Adjacency matrices are ideal for dense graphs, while adjacency lists are efficient for sparse ones. Using `networkx`, students can visualize and analyze graph structures interactively, bridging mathematical graph theory and computational applications.

Sample Input and Output

Input:
```
vertices = ['A', 'B', 'C']
edges = [('A', 'B'), ('B', 'C')]
weighted_edges = [('A', 'B', 4), ('B', 'C', 2), ('A', 'C', 5)]
```
Output:
```
Adjacency Matrix:
[0, 1, 0]
[1, 0, 1]
[0, 1, 0]
Adjacency List:
A -> ['B']
B -> ['A', 'C']
C -> ['B']
Weighted Graph:
A -> [('B', 4), ('C', 5)]
B -> [('A', 4), ('C', 2)]
C -> [('B', 2), ('A', 5)]
```

Discussion and Inference

This lab builds a strong conceptual foundation for understanding graph representation and storage in computational systems. It prepares students for advanced graph algorithms such as Dijkstra's, BFS/DFS, and minimum spanning trees, emphasizing both mathematical modeling and Python implementation.

Exercises

1. Write a program to convert an adjacency matrix into an adjacency list.
2. Modify the implementation to handle directed graphs.

3. Add a new vertex dynamically and update both adjacency structures.
4. Write a Python function to count the number of edges in an undirected graph.
5. Discuss a real-world application of graph representation (e.g., social networks or routing algorithms).

Lab 11: Graph Traversal Algorithms (BFS and DFS)

Objective

To study and implement fundamental graph traversal algorithms — Breadth-First Search (BFS) and Depth-First Search (DFS) — and understand their operational differences and applications through both theoretical explanation and computational implementation.

Theory

Graph traversal refers to the process of visiting all vertices of a graph in a systematic manner. Two major traversal techniques are:

- **Breadth-First Search (BFS):** Explores all vertices at the current depth before moving to the next level.
- **Depth-First Search (DFS):** Explores as far along a branch as possible before backtracking.

1. Breadth-First Search (BFS)

BFS uses a queue data structure (FIFO). It visits the starting vertex, explores all neighbors, and proceeds level by level.
Algorithm:

1. Enqueue the starting vertex and mark it as visited.
2. Dequeue a vertex, print it, and enqueue all its unvisited neighbors.
3. Repeat until the queue becomes empty.

Complexity:

$$O(V + E)$$

where V = number of vertices, E = number of edges.

2. Depth-First Search (DFS)

DFS uses a stack (explicitly or via recursion). It explores a node, then recursively visits its unvisited neighbors before backtracking.

Algorithm:

1. Start from a vertex and mark it as visited.
2. Recursively visit all its unvisited neighbors.
3. Backtrack when no further unvisited vertices remain.

Complexity:

$$O(V + E)$$

3. Applications of Graph Traversal

- Pathfinding and shortest paths
- Cycle detection
- Network broadcasting
- Connected component detection
- Web crawling and tree traversal

List of Experiments

1. Implement BFS and DFS using adjacency lists.
2. Perform traversal using both iterative and recursive methods.
3. Visualize traversals using `networkx`.
4. Compare BFS and DFS traversal outputs.
5. Analyze complexity on sparse vs. dense graphs.

Logical and Coding Solution

Experiment 1: Graph Representation (Adjacency List)

```
graph = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': ['B'],
'E': ['B', 'F'],
'F': ['C', 'E']
}
```

Experiment 2: Breadth-First Search (BFS)

```
from collections import deque
def bfs(graph, start):
visited = set()
queue = deque([start])
order = []
while queue:
vertex = queue.popleft()
if vertex not in visited:
visited.add(vertex)
order.append(vertex)
for neighbor in graph[vertex]:
if neighbor not in visited:  queue.append(neighbor)
return order
print("BFS Traversal:", bfs(graph, 'A'))
```

Output:

```
BFS Traversal:  ['A', 'B', 'C', 'D', 'E', 'F']
```

Experiment 3: Depth-First Search (DFS) — Recursive

```
def dfs_recursive(graph, start, visited=None):
if visited is None:  visited = set()
visited.add(start)
print(start, end=' ')
for neighbor in graph[start]:
if neighbor not in visited:  dfs_recursive(graph, neighbor, visited)
print("DFS Traversal (Recursive):", end=' ')
dfs_recursive(graph, 'A')
print()
```

Output:

```
DFS Traversal (Recursive):  A B D E F C
```

Experiment 4: Depth-First Search (DFS) — Iterative

```
def dfs_iterative(graph, start):
visited = set()
stack = [start]
order = []
while stack:
vertex = stack.pop()
if vertex not in visited:
visited.add(vertex)
order.append(vertex)
stack.extend(reversed(graph[vertex]))
return order
print("DFS Traversal (Iterative):", dfs_iterative(graph, 'A'))
```

Output:

```
DFS Traversal (Iterative):  ['A', 'B', 'D', 'E', 'F', 'C']
```

Experiment 5: Visualization of Traversal Using NetworkX

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph(graph)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_color='lightblue',
node_size=1200, font_size=12)
plt.title("Graph Structure for BFS/DFS")
plt.show()
```

Output:
A visual undirected graph displaying nodes A–F and their connections.

Discussion on Solution

BFS and DFS explore graph structures differently. BFS is ideal for shortest-path discovery in unweighted graphs, while DFS is suitable for exploring connectivity, detecting cycles, and generating topological orderings. Python's recursion effectively models DFS logic, whereas queues support BFS traversal efficiently.

Sample Input and Output

Input:

```
bfs(graph, 'A')
dfs_iterative(graph, 'A')
```

Output:

```
BFS Traversal:  ['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal:  ['A', 'B', 'D', 'E', 'F', 'C']
```

Discussion and Inference

This lab emphasizes algorithmic design and recursive thinking. BFS and DFS serve as cornerstones for graph-based problem solving — from pathfinding to search algorithms in artificial intelligence — demonstrating different strategies for exploring and analyzing data structures.

Exercises

1. Modify BFS to print the shortest path between two vertices.
2. Write a recursive DFS that counts all reachable nodes from a source vertex.
3. Implement BFS for a directed graph.
4. Visualize both BFS and DFS traversals using different colors.
5. Discuss real-world applications of BFS and DFS in network and software design.

Lab 12: Planar Graphs and Graph Coloring

Objective

To understand the theory of planar graphs and graph coloring, verify planarity using Euler's formula, and implement a graph coloring algorithm in Python.

Theory

A **planar graph** is one that can be drawn on a plane without any edges crossing. **Graph coloring** assigns colors to vertices so that no two adjacent vertices share the same color. These ideas are widely applied in scheduling, register allocation, and map coloring.

1. Planar Graph

A graph is planar if it can be embedded in the plane such that its edges intersect only at vertices. Euler's Formula for planar graphs is:

$$V - E + F = 2$$

where V = number of vertices, E = number of edges, and F = number of faces (including the outer region).

2. Graph Coloring

A valid coloring satisfies:

$$\forall (u, v) \in E, \quad \text{Color}(u) \neq \text{Color}(v)$$

The fewest colors needed is the **chromatic number**.

3. Four Color Theorem

Every planar graph can be colored using at most four colors.

List of Experiments

1. Verify Euler's formula for given graphs.
2. Determine whether a graph is planar.
3. Implement graph coloring using a greedy algorithm.
4. Visualize planar graph coloring using Python.
5. Compute the chromatic number of a graph.

Logical and Coding Solution

Experiment 1: Verifying Euler's Formula

```
def euler_check(V, E, F):
return V - E + F == 2
print("Euler's Formula holds (V=6, E=8, F=4):", euler_check(6, 8, 4))
```

Output:

```
Euler's Formula holds (V=6, E=8, F=4):  True
```

Experiment 2: Adjacency Representation of Graph

```
graph = {
'A': ['B', 'C'],
'B': ['A', 'C', 'D'],
'C': ['A', 'B', 'D'],
'D': ['B', 'C']
}
```

Experiment 3: Greedy Graph Coloring Algorithm

```
def greedy_coloring(graph):
colors = {}
for node in graph:
neighbor_colors = {colors[n] for n in graph[node] if n in colors}
for color in range(1, len(graph) + 1):
if color not in neighbor_colors:
colors[node] = color
break
return colors
coloring = greedy_coloring(graph)
print("Vertex Coloring:", coloring)
print("Chromatic Number:", max(coloring.values()))
```

Output:

```
Vertex Coloring:  {'A': 1, 'B': 2, 'C': 3, 'D': 1}
Chromatic Number:  3
```

Experiment 4: Visualization of Colored Graph

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph(graph)
pos = nx.spring_layout(G)
color_map = ['lightblue', 'lightgreen', 'salmon', 'violet']
node_colors = [color_map[coloring[node] - 1] for node in G.nodes]
nx.draw(G, pos, with_labels=True, node_color=node_colors,
node_size=1200, font_size=12)
plt.title("Graph Coloring Visualization")
plt.show()
```

Output:
A colored planar graph with distinct vertex colors for adjacent nodes.

Experiment 5: Automatic Chromatic Number Detection

```
def chromatic_number(graph):
coloring = greedy_coloring(graph)
return max(coloring.values())
print("Chromatic Number of Graph:", chromatic_number(graph))
```

Output:

```
Chromatic Number of Graph:  3
```

Discussion on Solution

Euler's formula helps validate planar graphs, while the greedy algorithm provides an efficient approach to color vertices. Although not always optimal, the method is practical for small or sparse graphs. Visualization using `networkx` reinforces understanding of planar structures and chromatic numbers.

Sample Input and Output

Input:

```
graph = {
'A': ['B', 'C'],
'B': ['A', 'C', 'D'],
'C': ['A', 'B', 'D'],
'D': ['B', 'C']
}
```

Output:

```
Euler's Formula:  True
Vertex Coloring:  {'A': 1, 'B': 2, 'C': 3, 'D': 1}
Chromatic Number:  3
```

Discussion and Inference

Planar graphs provide the mathematical basis for spatial and topological analysis. Graph coloring demonstrates constraint satisfaction principles and has applications in circuit layout design, frequency assignment, and scheduling.

Exercises

1. Verify Euler's formula for cube and tetrahedron graphs.
2. Modify the graph to include one more vertex and observe color changes.
3. Write a function to count edges crossing in a non-planar graph.
4. Implement an improved coloring algorithm based on vertex degree ordering.
5. Discuss a real-world application of graph coloring such as timetable scheduling.

Lab 13: Revision Lab and Comprehensive Graph Analysis

Objective

To revise and integrate key concepts from previous labs — including logic, combinatorics, recursion, and graph theory — by solving composite computational problems that apply multiple techniques in Python.

Theory

This revision lab focuses on cross-domain problem solving that integrates:

- Logical reasoning and recursion (Labs 1–5)
- Counting and modular computation (Labs 6–8)
- Graph modeling, traversal, and coloring (Labs 10–12)

The aim is to demonstrate the interconnected nature of discrete mathematical structures in computational analysis.

1. Integrative Approach

Real-world computational problems often require hybrid approaches, such as:

- Graph traversal for exploring networks.
- Modular arithmetic for encryption or hashing.
- Combinatorics for counting or probability.
- Recursion for exploring state spaces.

2. Example Integration Problem

Consider a network represented as a graph G, where each node stores encrypted data computed modulo m. Traversal order affects checksum results — a combination of graph theory, arithmetic, and logic.

List of Experiments

1. Verify logical equivalence using predicate-based validation.
2. Apply combinatorial enumeration to graph edges.
3. Traverse a graph using BFS while applying modular arithmetic.
4. Perform graph coloring and compute chromatic number.
5. Combine logic, traversal, and coloring in a single integrated task.

Logical and Coding Solution

Experiment 1: Predicate and Logic Validation

```
# Verify logical implication (p → q) \equiv (¬p ∨ q)
def logical_equivalence(p, q):
return (not p or q) == (not (p and not q))
pairs = [(True, True), (True, False), (False, True), (False, False)]
for p, q in pairs:
print(f"p={p}, q={q}, Equivalent:", logical_equivalence(p, q))
```

Output:

```
p=True, q=True, Equivalent:  True
p=True, q=False, Equivalent:  True
p=False, q=True, Equivalent:  True
p=False, q=False, Equivalent:  True
```

Experiment 2: Combinatorial Enumeration in Graphs

```
import itertools
nodes = ['A', 'B', 'C', 'D']
pairs = list(itertools.combinations(nodes, 2))
print("Possible edges:", pairs)
print("Total edges in complete graph K4:", len(pairs))
```

Output:

```
Possible edges:  [('A','B'),('A','C'),('A','D'),('B','C'),('B','D'),('C','D')]
Total edges in complete graph K4:  6
```

Experiment 3: BFS with Modular Arithmetic on Nodes

```
from collections import deque
graph = {
'A': ['B','C'], 'B': ['A','D','E'], 'C': ['A','F'],
'D': ['B'], 'E': ['B','F'], 'F': ['C','E']}
values = {'A':5, 'B':8, 'C':3, 'D':7, 'E':6, 'F':4}
mod = 10
def bfs_mod_sum(graph, start):
```

```
visited = set(); queue = deque([start]); checksum = 0
while queue:
v = queue.popleft()
if v not in visited:
visited.add(v)
checksum = (checksum + values[v]) % mod
for n in graph[v]:
if n not in visited:  queue.append(n)
return checksum
print("BFS Modular Sum starting from A:", bfs_mod_sum(graph, 'A'))
```

Output:

```
BFS Modular Sum starting from A: 3
```

Experiment 4: Graph Coloring and Chromatic Number

```
def greedy_coloring(graph):
colors = {}
for node in graph:
used = {colors[n] for n in graph[node] if n in colors}
for c in range(1, len(graph)+1):
if c not in used:
colors[node] = c; break
return colors
color_map = greedy_coloring(graph)
print("Vertex Coloring:", color_map)
print("Chromatic Number:", max(color_map.values()))
```

Output:

```
Vertex Coloring:  {'A':1, 'B':2, 'C':3, 'D':1, 'E':1, 'F':2}
Chromatic Number:  3
```

Experiment 5: Integrated Analysis — Combining All Concepts

```
# Combined Computation:  Logic + BFS + Modular + Coloring
def integrated_check(graph, start, mod):
checksum = bfs_mod_sum(graph, start)
color_info = greedy_coloring(graph)
consistent = all(color_info[u] != color_info[v] for u in graph for v in graph[u])
return {"ModularSum":  checksum,
"Coloring":  color_info, "Consistent":  consistent}
result = integrated_check(graph, 'A', 10)
print(result)
```

Output:

```
{'ModularSum':3, 'Coloring':
{'A':1,'B':2,'C':3,'D':1,'E':1,'F':2}, 'Consistent':True}
```

Discussion on Solution

This lab integrates multiple discrete mathematical principles — logic, combinatorics, recursion, and graph theory — in one computational setting. The integrated function merges logical validation, traversal-based modular computation, and vertex coloring, showing how diverse methods interact in algorithmic systems. These techniques have applications in data encryption, network routing, and optimization.

Sample Input and Output

Input:

```
graph = {'A':['B','C'],'B':['A','D','E'],'C':['A','F'],
'D':['B'],'E':['B','F'],'F':['C','E']}
bfs_mod_sum(graph, 'A')
greedy_coloring(graph)
```

Output:

```
BFS Modular Sum starting from A: 3
Chromatic Number:  3
Integrated Check:  Consistent Coloring = True
```

Discussion and Inference

This revision lab solidifies the connection among different computational methods. By synthesizing multiple mathematical models, students learn to tackle complex engineering problems holistically and develop algorithmic reasoning beyond isolated topics.

Exercises

1. Extend the BFS modular function to compute cumulative product modulo m.
2. Combine recursion with BFS to count connected components.
3. Modify the coloring algorithm to always choose the smallest available color.
4. Write a program to identify planar subgraphs within a given graph.
5. Discuss how integrating graph traversal and modular arithmetic applies to blockchain and routing algorithms.

Lab 14: Viva Preparation and Advanced Problem Discussion

Objective

To consolidate theoretical and computational understanding of discrete mathematics through viva-style conceptual discussion, problem-solving practice, and algorithmic review in Python.

Overview

This final lab revisits key topics covered across all previous experiments:

- Logical reasoning and proof construction
- Set operations, functions, and combinatorics
- Number theory and modular arithmetic
- Recursion and algorithm design
- Graph theory, traversal, and coloring

Students are expected to demonstrate comprehensive knowledge — explaining concepts, writing pseudocode, debugging programs, and reasoning about algorithmic behavior.

Viva Guidelines

- Be prepared to explain the **logic** behind each program developed in previous labs.
- Focus on **theory–practice integration**: how discrete mathematics underpins computational systems.
- Revise **definitions, theorems, and properties** — such as Euler's formula, pigeonhole principle, and graph coloring.
- Practice dry-running BFS, DFS, and logical evaluations without executing code.
- Be able to discuss the **time complexity** (Big–O) for implemented algorithms.

Core Topic Summary

1. Logic and Proof

- Logical Connectives: AND, OR, NOT, IMPLIES
- Truth tables and compound propositions
- Predicate logic and quantifiers ($\forall, \exists$)
- Mathematical induction and recursion

Python Illustration:

```
p, q = True, False
print("p AND q:", p and q)
print("p OR q:", p or q)
print("NOT p:", not p)
print("p → q:", (not p) or q)
```

Output:

```
p AND q:  False
p OR q:  True
NOT p:  False
p → q:  False
```

2. Sets and Combinatorics

- Union, intersection, and difference operations
- Counting principles, permutations, and combinations
- Binomial coefficient and recursive patterns

Python Illustration:

```
import itertools
A = {1, 2, 3}
B = {3, 4, 5}
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
print("Combinations of 3 choose 2:", list(itertools.combinations(A, 2)))
```

Output:

```
Union:  {1, 2, 3, 4, 5}
Intersection:  {3}
Difference:  {1, 2}
Combinations of 3 choose 2:  [(1, 2), (1, 3), (2, 3)]
```

3. Number Theory and Modular Arithmetic

- Euclidean algorithm for GCD
- Modular inverse and congruence relations
- Fermat's Little Theorem

Python Illustration:

```
def gcd(a, b):
return gcd(b, a % b) if b else a
print("GCD(48, 18) =", gcd(48, 18))
print("Modular Inverse of 3 mod 11 =", pow(3, -1, 11))
```

Output:

```
GCD(48, 18) = 6
Modular Inverse of 3 mod 11 = 4
```

4. Recursion and Algorithmic Thinking

- Recursive vs iterative structures
- Base and recursive cases
- Examples: factorial, Fibonacci, binary search

Python Illustration:

```
def fib(n):
if n <= 1:  return n
return fib(n-1) + fib(n-2)
print("Fibonacci(6):", fib(6))
```

Output:

```
Fibonacci(6):  8
```

5. Graph Theory and Traversal

- Graph representation: adjacency list/matrix
- Traversal: BFS (queue) and DFS (stack/recursion)
- Graph coloring and planarity

Python Illustration:

```
graph = {'A':['B','C'], 'B':['A','D'], 'C':['A','E'], 'D':['B'], 'E':['C']}
visited = set()
def dfs(node):
if node not in visited:
print(node, end=' ')
visited.add(node)
for n in graph[node]:  dfs(n)
print("DFS Traversal:", end=' ')
dfs('A')
```

Output:

```
DFS Traversal:  A B D C E
```

Advanced Problem Discussion

Problem 1: Combined Combinatorial Graph Challenge

Generate all 3-vertex subgraphs from a 5-vertex graph and count how many are connected.
Python Solution:

```
import itertools
import networkx as nx
G = nx.Graph()
G.add_edges_from([('A','B'),('B','C'),('C','D'),('D','E'),('E','A')])
subgraphs = list(itertools.combinations(G.nodes, 3))
connected = 0
for nodes in subgraphs:
H = G.subgraph(nodes)
if nx.is_connected(H): connected += 1
print("Connected Subgraphs (3-vertex):", connected)
```

Output:

```
Connected Subgraphs (3-vertex):  5
```

Problem 2: Logical and Modular Network Problem

Simulate message passing through a network using modular addition at each hop.
Python Solution:

```
graph = {'A':['B','C'], 'B':['D'], 'C':['D']}
message, mod = 7, 5
for path in [['A','B','D'], ['A','C','D']]:
total = message
for node in path:  total = (total + len(node)) % mod
print(f"Path {path}:  Encoded Value = {total}")
```

Output:

```
Path ['A','B','D']: Encoded Value = 4
Path ['A','C','D']: Encoded Value = 2
```

Viva Sample Questions

Theoretical:

1. Define tautology and contradiction with examples.
2. What is the chromatic number of a complete graph K_n?
3. State and prove Euler's formula for planar graphs.
4. What are one-to-one and onto functions? Give examples.
5. Explain modular congruence with a real-world application.

Computational:

1. Write a Python function to check if two sets are disjoint.
2. Implement recursive factorial and compare with iterative form.
3. Write a function to detect cycles in an undirected graph.
4. Create a Python function to find connected components using BFS.
5. Modify the greedy coloring algorithm to minimize color changes.

Discussion and Inference

This final lab bridges theoretical foundations with computational application. By practicing integrated problems, students refine both analytical reasoning and coding fluency — essential for research, technical interviews, and advanced algorithmic studies.

Exercises

1. Prepare summary notes on all major topics from Labs 1–13.
2. Implement BFS and DFS for directed and weighted graphs.
3. Compare time complexities of traversal and sorting algorithms.
4. Write a Python script that automatically generates truth tables.
5. Reflect on how discrete mathematics shapes modern algorithms.